Regularising deep networks

CIFAR-10

LECTURE 10 | GÉRON CH. 11

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Lecture 9 took a twenty-layer network that sat at chance and repaired it:

- the gradient shrank by a constant factor per layer — a straight line on a log axis — because one layer multiplies the variance by $n \cdot \mathrm{Var}(w)$
- He initialisation and ReLU set that factor to one; batch normalisation, clipping and an optimiser were measured on top
- the ablation ladder: one change, ReLU with He, did more than the rest together, and the last rung — dropout — made things *worse*

> **FAILURE CONDITION — A NETWORK THAT LEARNS THE TRAINING SET BY HEART**
>
> Train the repaired network for longer and it keeps improving — on the images it trains on. On the images it does not, it stops at forty percent and stays there. Today: why, what the cures are, which of them pay, and what that last rung was.

Today

1. **The gap** — the training curve climbs, the validation curve stalls; the gap named as variance
2. **The mathematics** — dropout as an average over 2^n thinned networks, and the rescaling that makes it one; weight decay as a shrinkage applied before every step
3. **The method** — five cures: early stopping, weight decay, dropout, augmentation, label smoothing; each measured alone, then stacked on the ladder
4. The failure condition that cannot be seen from the number: a knob tuned on the test set
5. **Design exercise** — the lesion check inside a dermatology app, in nine steps, with the pitfalls and how to address them

Open Lecture 10 in Colab

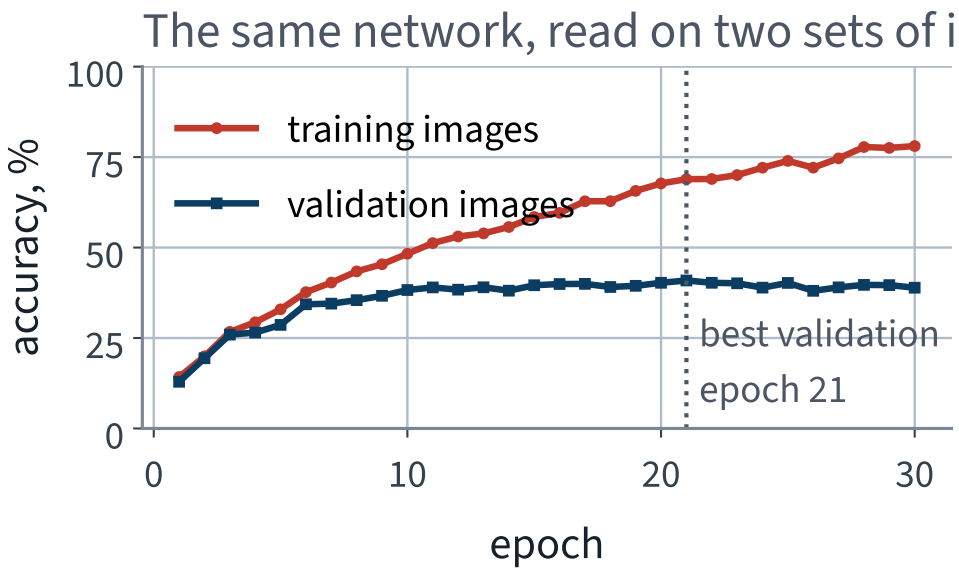
The specification

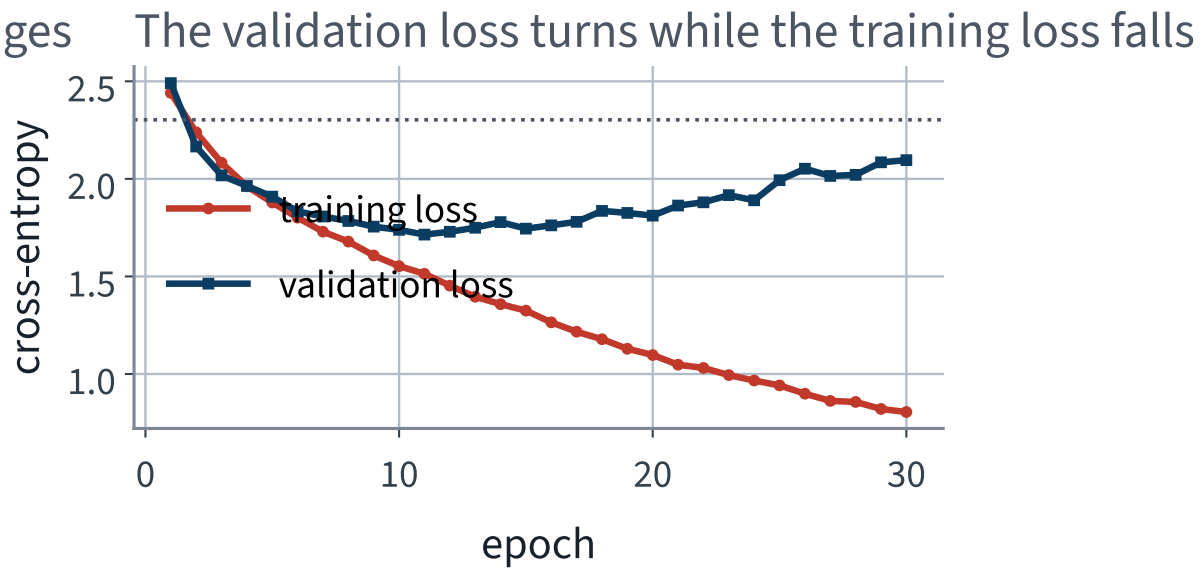
> **WHAT WE ARE ASKING FOR**
>
> The network Lecture 9 left us: 3,072 inputs, **20 hidden layers** of 100 units, each `Linear → BatchNorm → ReLU`, He initialisation, a linear head of 10 outputs. Cross-entropy, Adam at 0.001, batches of 128, clipping at 1.0. The same 10,000 training images and 5,000 validation images, the same seed — and **30 epochs** instead of 20.

No learning-rate schedule this time: a schedule fixes the number of steps in advance, and one of today's cures is to stop when the validation set says so.

One new column in the log: the accuracy on the **training** images after every epoch, measured in evaluation mode on all 10,000 of them, beside the validation accuracy.

The same network, read on two sets of images





✗ **The network is still learning. It is learning *these* ten thousand images.**

Reading it

Epoch	Training accuracy	Validation accuracy
20 — where Lecture 9 stopped	67.71%	40.26%
✓ 21 — the best validation epoch	68.90%	**40.90% ✓**
✗ 30 — the last: a gap of 39.15 points	78.05%	**✗ 38.90%**

- Nine more epochs bought ten points on the training images and lost two on the validation images
- The validation *loss* turned earlier than the validation accuracy: lowest at epoch 11, at 1.7145, and 2.0959 by the end. Confident wrong answers cost more than they used to
- Test accuracy of the last weights: **40.11%**. Of the epoch-21 weights: **41.46%**. Keep that pair for later

Name the gap: variance

The error on new images that comes from fitting the *particular* sample — which pixels this frog has, rather than what a frog looks like. The arithmetic of why the network can:

Free parameters	504,210
Training images	10,000
Parameters per training image	50.4
Bits of label information in the training set, $10{,}000 \cdot \log_2 10$	33,219

✓ Half a million weights can store thirty thousand bits many times over. Given enough epochs, they will.

Lecture 5 met the same thing in a tree grown to purity, and averaged it away with a forest. A network has no cheap forest — but one of today's cures turns out to be one.

Every cure is a constraint

Cure	What it constrains	The knob
Early stopping	how far the fit runs	the epoch, chosen on validation
Weight decay	the size of the weights	λ
Dropout	which units may rely on which	p, and which layers
Data augmentation	the function: it must give the same answer on an image and its copies	which transforms
Label smoothing	the target: no logit is asked to be infinite	ε

Each one trades a little bias — the constraint may be wrong — for less variance. Each has a knob, and every knob is set on the validation set. The last slide before the design exercise is about what happens when it is not.

Dropout: the mechanism

At every training step, each hidden unit is set to zero with probability p, independently. At test time nothing is dropped.

```
layers += [nn.Linear(prev, 100), nn.BatchNorm1d(100), nn.ReLU()]
if dropout and i >= DEPTH - top:        # the top hidden layers only
    layers.append(nn.Dropout(dropout))  # p: the DROP rate, not the keep rate

net.train()   # masks sampled, kept units scaled by 1/(1 - p)
net.eval()    # no masks, no scaling: the full network
```

Two things to settle before it can be used: what the network without masks computes, and why the kept units are rescaled. Both are the derivation.

A layer of n units is 2^n thinned networks

A mask $\mathbf{m} \in \{0,1\}^n$ chooses which units survive. Each choice is a different, smaller network — a *thinned* network — and all of them **share the same weights**. One training step samples one mask, runs that thinned network forward and backward, and updates the shared weights.

$$2^n \text{ thinned networks;} \qquad n = 100 : \; 2^{100} \approx 10^{30.1}$$

MORE NETWORKS THAN TRAINING STEPS, BY TWENTY-SEVEN ORDERS OF MAGNITUDE

So almost every thinned network is trained zero times. What makes this work is that they share weights — a weight that helps one thinned network helps most of them — and the question becomes what the full network, run without masks, has to do with their average.

The expected pre-activation under the mask

One unit of the next layer sees $z = \sum_j w_j a_j$ from activations a_j. Under a mask $m_j \sim \text{Bernoulli}(1-p)$, independent across j,

$$\tilde{z} = \sum_j m_j \, w_j \, a_j, \qquad \mathbb{E}[\tilde{z}] = \sum_j \mathbb{E}[m_j] \, w_j a_j = (1-p) \, z$$

by linearity of expectation, with the a_j and w_j held fixed. Train with masks and run without them, and every pre-activation is $1/(1-p)$ times larger than anything the weights were trained on.

$$\tilde{z} = \frac{1}{1-p} \sum_j m_j \, w_j \, a_j \qquad \Longrightarrow \qquad \mathbb{E}[\tilde{z}] = z$$

INVERTED DROPOUT: DIVIDE THE KEPT ACTIVATIONS BY $1 - P$ IN TRAINING, AND THE FULL NETWORK AT TEST TIME COMPUTES THE EXPECTATION

The expectation is preserved. The variance is not.

For one term, $m_j^2 = m_j$ because $m_j \in \{0, 1\}$, so

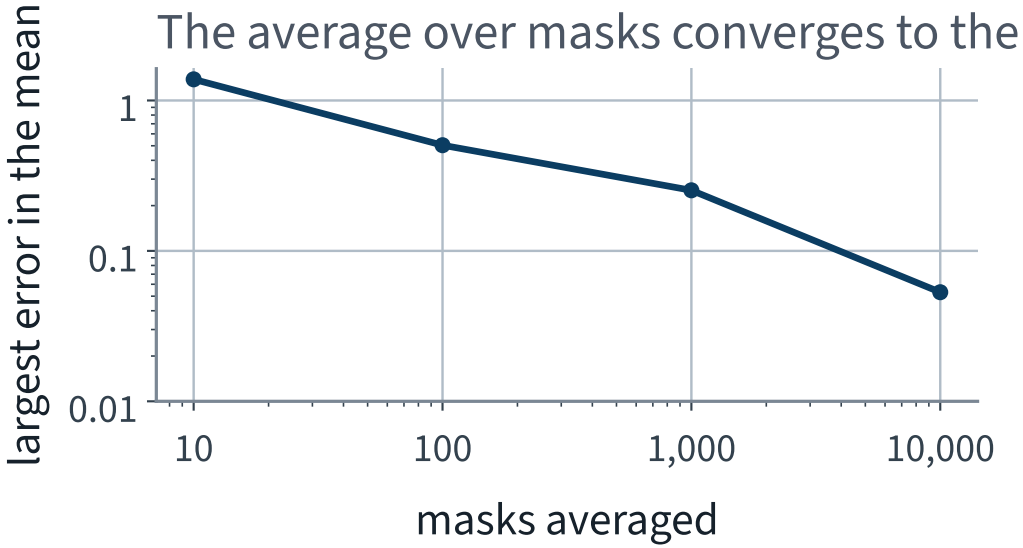
$$\mathbb{E}\left[\left(\frac{m_j a_j}{1-p}\right)^2\right] = \frac{\mathbb{E}[m_j]\, a_j^2}{(1-p)^2} = \frac{a_j^2}{1-p}$$

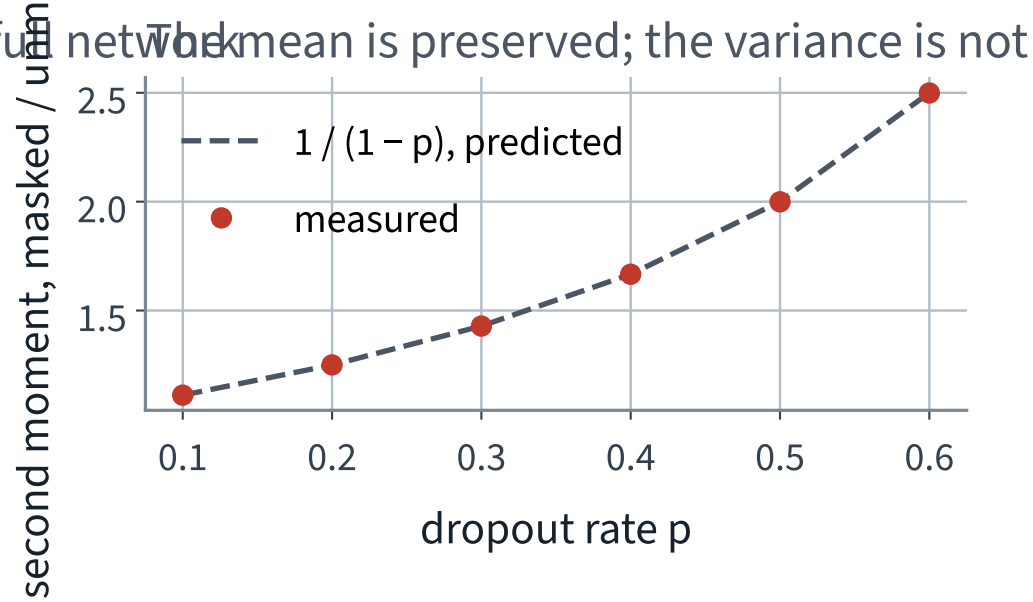
Every dropout layer multiplies the second moment of the signal passing through it by $1/(1-p)$. The expectation is right; the spread around it is not, and on a deep stack it compounds:

Dropout on	p	Factor on the second moment, $(1-p)^{-L}$
one layer	0.5	2
✗ all twenty layers	0.1	✗ 8.23
✗ all twenty layers	0.5	✗ 1,048,576

✓ Lecture 9's argument about ρ, again: a factor applied twenty times is a different thing from a factor applied once. Dropout goes on the top layer or two.

The derivation, measured on the trained network





On the last hidden layer, 20,000 masks at $p = 0.5$: the average is within 0.0433 of z on pre-activations of mean size 1.999, and the second moment is 2.000 times the unmasked one.

What the result buys you

- **An ensemble for free.** The network run without masks computes the expectation over 2^n thinned networks — an average of models that share weights, Lecture 5's cure for variance without the forest's bill
- **No unit can rely on a particular neighbour**, because the neighbour is absent half the time; the features it learns are individually useful rather than co-adapted
- **Keep the masks on at test time** and you have the ensemble's spread as well as its mean: Monte Carlo dropout, a measure of how sure the network is, for the price of T forward passes
- **Where to put it:** the top layer or two, at $p \in [0.2, 0.5]$, because each dropout layer multiplies the second moment by $1/(1-p)$ and the factor compounds with depth

Weight decay: the step, written out

Add $\frac{\lambda}{2}\|\theta\|^2$ to the loss. Its gradient is $\lambda\theta$, so one gradient step is

$$\theta \leftarrow \theta - \eta\left(\nabla L + \lambda\theta\right) = \left(1 - \eta\lambda\right)\theta - \eta\nabla L$$

$$\theta \leftarrow \left(1 - \eta\lambda\right)\theta - \eta\nabla L$$

FIRST SHRINK EVERY WEIGHT BY THE FACTOR $1 - H\Lambda$, THEN TAKE THE USUAL STEP

The ℓ_2 penalty and the per-step shrinkage are the same thing — for gradient descent. With $\eta = 0.001$ and $\lambda = 0.2$ the factor is 0.9998 per step; over the 2,370 steps of a thirty-epoch run a weight that receives no gradient shrinks to 0.62 of itself. A weight that earns its size keeps it; one that does not, decays.

Where AdamW puts it

Adam divides each coordinate's step by a running estimate of its gradient's size. Put $\lambda\theta$ *into* the gradient and it is divided too:

	The update	What λ does
✗ `Adam(weight_decay=λ)` — **coupled**	$\theta \leftarrow \theta - \eta \dfrac{\widehat{m}(\nabla L + \lambda\theta)}{\sqrt{\widehat{v}} + \epsilon}$	normalised away: every weight moves by about η, whatever its size
✓ `AdamW(weight_decay=λ)` — **decoupled**	$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta \dfrac{\widehat{m}(\nabla L)}{\sqrt{\widehat{v}} + \epsilon}$	the shrinkage, as derived

One step with a **zero** loss gradient, $\eta = 0.001$, $\lambda = 0.2$, so only the decay moves anything:

	0.5 becomes	0.0005 becomes
SGD, AdamW	0.4999	0.0004999
✗ Adam, coupled	0.4990	**✗ −0.0005**

Cure 1 — early stopping

```
if h["val_acc"][-1] > h["best_val"]:           # chosen on VALIDATION
    h["best_val"]   = h["val_acc"][-1]
    h["best_epoch"] = ep + 1
    h["best_state"] = {k: v.detach().clone()      # a COPY: training goes on
                       for k, v in net.state_dict().items()}
...
best.load_state_dict(h["best_state"])
h["test_at_best"] = accuracy(best, Xt, yt)          # what you ship, scored once
```

- Keep the **best epoch**, not the last: the curve on slide 5 says the last is worse
- Two rules. The epoch is chosen on the validation set. What is reported is the test accuracy of the *kept* weights — not the validation accuracy that chose them, which is optimistic by construction

Early stopping, measured

Weights shipped	Test accuracy
✗ epoch 30, the last	40.11%
✓ epoch 21, the best on validation	**41.46% ✓**

✓ +1.35 points, and nine epochs of training not spent memorising.

It did not close the gap: at epoch 21 the training accuracy was already 68.90%. Early stopping stops the damage; it does not constrain the model. The validation loss had turned ten epochs earlier still, at epoch 11: a reasonable rule would have stopped there, and the two rules land within a point of each other. The validation set decides; the test set is read once.

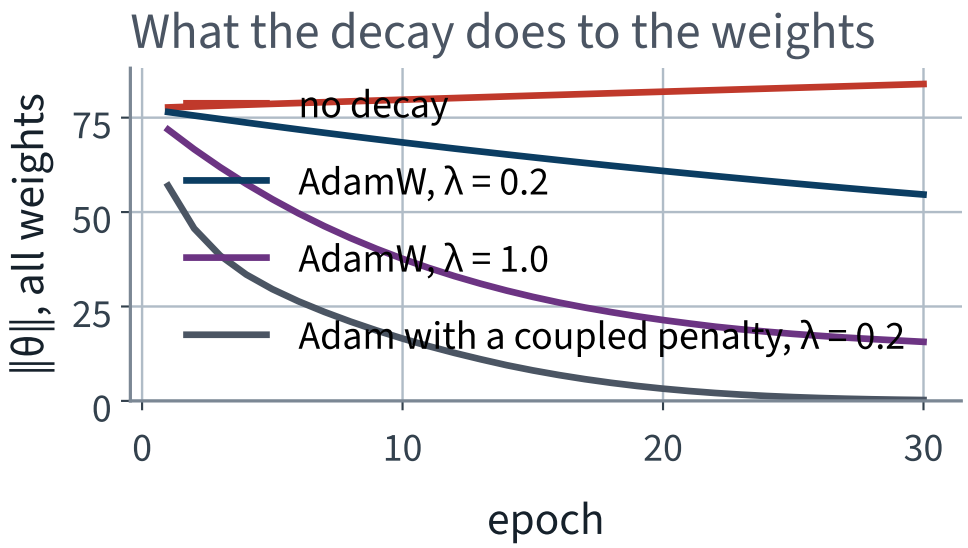
Cure 2 — weight decay, $\lambda = 0.2$

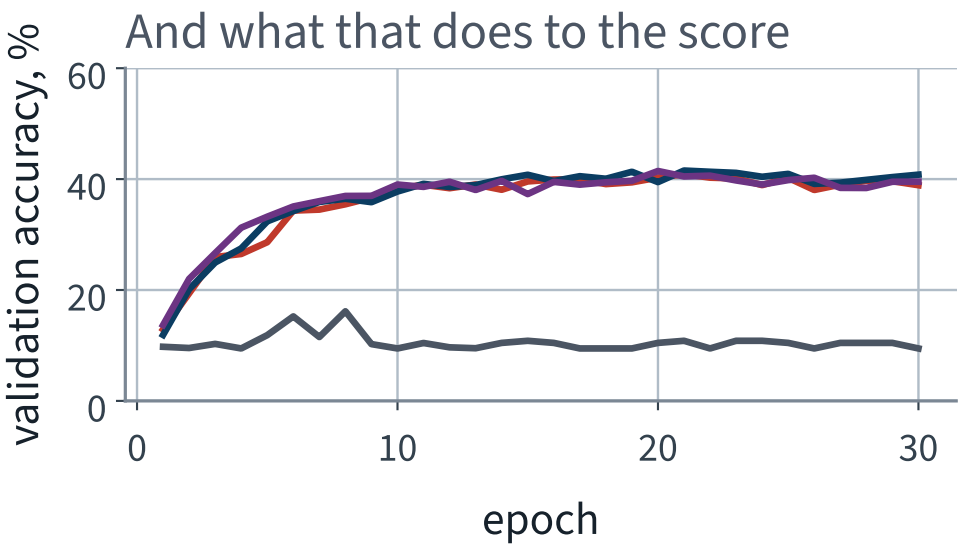
```
optim = torch.optim.AdamW(net.parameters(), lr=1e-3, weight_decay=0.2)
```

	$\|\theta\|$ after epoch 30	Training	Validation	Gap	Test
no decay	83.9	78.05%	38.90%	✗ **39.15**	40.11%
✓ **AdamW, $\lambda = 0.2$**	**54.6** ✓	75.19%	40.80%	34.39	41.34%

The weights are a third smaller, the gap is five points narrower, the test accuracy is a point higher — a point being about the seed-to-seed spread, as the ladder will show. Without decay the norm *grew* over the run, from 77.7 after the first epoch to 83.9.

What the decay does to the weights, and to the score





Same λ, two spellings: AdamW shrinks the weights; Adam with a coupled penalty erases them.

Failure condition — the penalty inside Adam

ADAM(WEIGHT_DECAY=0.2) ON THIS NETWORK

Test accuracy **10.00%** — chance — and $\|\theta\| = 0.2$ after thirty epochs, against 83.9 without decay. The normalised penalty moves every weight by about η towards zero at every step whatever its size, and five hundred thousand weights of size 0.1 reach zero in a hundred steps.

- Nothing raises. The loss sits at $\ln 10$ and the run completes
- The same λ in `AdamW` is the second row of the previous table. The optimiser name is the whole difference
- A value of λ that is right for one optimiser is wrong by orders of magnitude for the other; the knob does not transfer

Cure 3 — dropout, and where it goes

```
net = make_net(dropout=0.5, top=1)      # after the top hidden layer only
net = make_net(dropout=0.1, top=20)     # Lecture 9's last rung: every layer
```

	Train, epoch 20	Val, epoch 20	Train, epoch 30	Val, epoch 30	Gap	Test
none	67.71%	40.26%	78.05%	38.90%	**✗ 39.15**	40.11%
✓ 0.5 on the top layer	—	—	74.62%	41.24%	33.38	**41.56% ✓**
✗ 0.1 on every layer	39.09%	**✗ 33.90%**	47.39%	37.08%	10.31	**✗ 37.55%**

On the top layer: a narrower gap and a point and a half on the test set — a modest cure on a network this narrow. On every layer: the narrowest gap on the slide, bought by not learning.

Lecture 9’s last rung, explained

That ladder stopped at epoch 20 and read dropout 0.1 on every layer as a loss of seven points. Now the whole curve is in view:

- at epoch 20 it stood at 33.90% on validation against 40.26% without dropout — the row Lecture 9 printed
- its *training* accuracy at that epoch was 39.09%: it was not overfitting, it was barely fitting
- the second moment of the signal was multiplied by 8.23 on the way up twenty layers of masks, and the network spent its epochs learning through that noise

✓ Dropout on every layer of a deep, narrow stack is not a regulariser. It is a slower learner with a narrower gap.

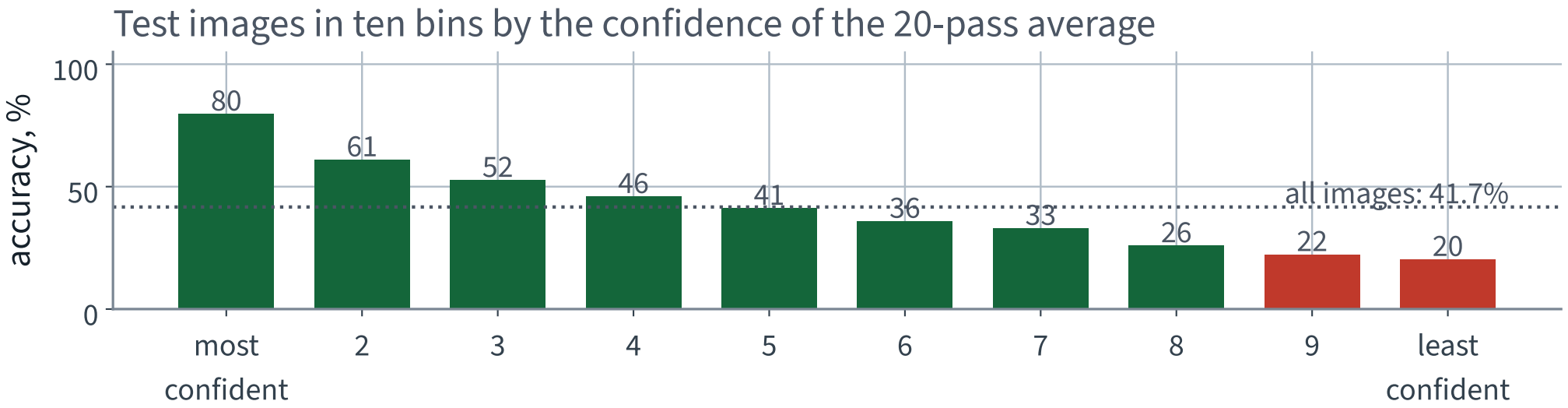
Lecture 9 read that rung as a regulariser added to a network that was not overfitting. The training column, which that deck did not log, says it was overfitting already; the rung failed for the reason on the previous slide.

Monte Carlo dropout: keep the masks on

```
net.eval()                                  # BatchNorm: running statistics, frozen
for m in net:
    if isinstance(m, nn.Dropout):
        m.train()                           # Dropout: masks on
probs = sum(F.softmax(net(X), 1) for _ in range(T)) / T   # T = 20 passes
pred, conf = probs.argmax(1), probs.max(1).values
```

- T thinned networks, averaged: an ensemble whose members were trained together
- The largest averaged probability is the network's confidence, and it can be *acted on*: answer when confident, refer the rest
- Only the dropout modules go back to training mode. Switch the whole network and batch normalisation starts using batch statistics, and the spread measures the batches, not the masks

Confidence that means something



Masks off: 41.81%. Twenty passes averaged: 41.69% — no better. But the top decile by confidence is right 79.6% of the time and the bottom decile 20.2%; answer the 80% most confident and the accuracy on what is answered is 46.81%.

Cure 4 — data augmentation

A photograph flipped left-to-right, or shifted by two pixels, is a photograph of the same thing. Train on random copies and the network is told so: the function it fits has to give the same answer on an image and its copies, which rules out the part of the hypothesis space that was memorising pixel positions.

```
def augment(xb, g):                          # on standardised rows, as images
    img = xb.view(n, 32, 32, 3)
    flip = torch.rand(n, generator=g) < 0.5
    img = torch.where(flip[:, None, None, None], img.flip(2), img)
    padded = F.pad(img.permute(0, 3, 1, 2), (2, 2, 2, 2))    # 36 x 36
    ...                                      # a random 32 x 32 window of it
xb = augment(Xf[idx], g)                     # a FRESH copy of every batch, every step
```

Which transforms is a statement about the data: a flipped digit is a different digit, a flipped frog is a frog.

One training image, seven random copies

One training image of class “ship” and seven random copies: a flip, and a shift of up to two pixels



The network never sees the same batch twice. It cannot memorise what it never sees twice.

Augmentation, measured

	Training	Validation	Gap	Test	Same answer on an image and its flip
none	78.05%	38.90%	✗ 39.15	40.11%	52.91%
✓ flips and shifts	53.26%	44.58% ✓	8.68 ✓	44.88% ✓	71.77% ✓

✓ +4.77 points on the test set, and the gap cut from thirty-nine points to nine. The one cure that did more than narrow the gap.

The last column is the constraint, measured: without augmentation the network changes its answer on half the test images when they are flipped — it had learned where the pixels were. With it, the flip is nearly invisible to it.

Failure condition — the evaluator's transform

> **THE SAME TRANSFORM ON THE TEST LOADER**
>
> Score the augmented network on a random copy of each test image instead of the image: 43.64% instead of 45.15%. The network did not change; the task did. A shifted test image with a two-pixel border is not a photograph anyone will hand the system.

- Augmentation lives in the training path, and only there. Validation and test images are never transformed
- The usual way to get this wrong is a single `transform` object shared by both loaders. The usual symptom is nothing: a plausible number, a little low
- The network trained without augmentation, on the same copies: 37.48%. Both numbers are honest answers to a question nobody asked

Cure 5 — label smoothing, $\varepsilon = 0.1$

$$y_k = (1 - \varepsilon)\,[k = \text{true}] + \frac{\varepsilon}{10}$$

THE TARGET: 0.91 ON THE TRUE CLASS, 0.01 ON EACH OF THE OTHER NINE

```
lossf = nn.CrossEntropyLoss(label_smoothing=0.1)
```

	Training loss, epoch 30	Training	Validation	Gap	Test
one-hot targets	0.8052	78.05%	38.90%	✗ **39.15**	40.11%
✓ **smoothed,** $\varepsilon = 0.1$	1.1662	76.32%	39.92%	36.40	41.18%

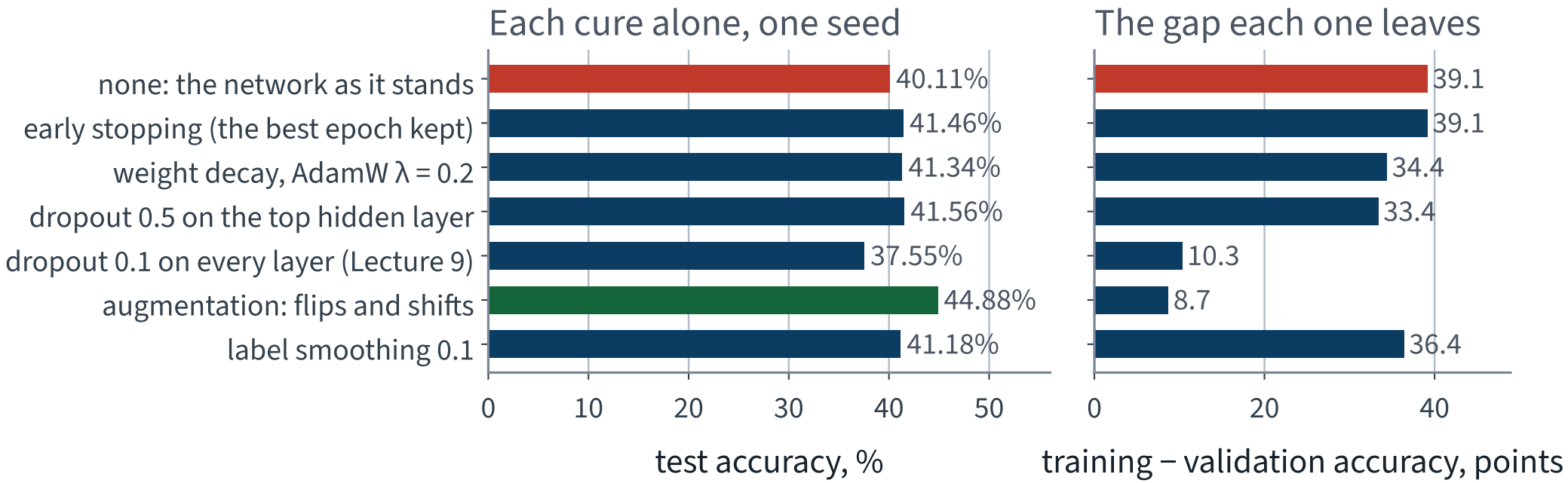
The training loss is *higher* by construction: the target can no longer be reached by driving one logit to infinity, which was the mechanism that kept the training loss falling after the validation loss had turned. A point on the test set; the gap barely moves.

Each cure alone, one seed

Cure	Training	Validation	Gap	Test
✗ none: the network as it stands	78.05%	38.90%	**✗ 39.15**	40.11%
early stopping, the best epoch kept	78.05%	38.90%	39.15	41.46%
weight decay, AdamW $\lambda = 0.2$	75.19%	40.80%	34.39	41.34%
dropout 0.5 on the top layer	74.62%	41.24%	33.38	41.56%
dropout 0.1 on every layer	47.39%	37.08%	10.31	**✗ 37.55%**
✓ augmentation: flips and shifts	53.26%	44.58%	8.68	**44.88% ✓**
label smoothing 0.1	76.32%	39.92%	36.40	41.18%

Every cure narrows the gap. One of them raises the test accuracy by more than a point. Narrowing the gap and generalising better are not the same thing: a network that learns nothing has no gap at all.

The same table, as a picture



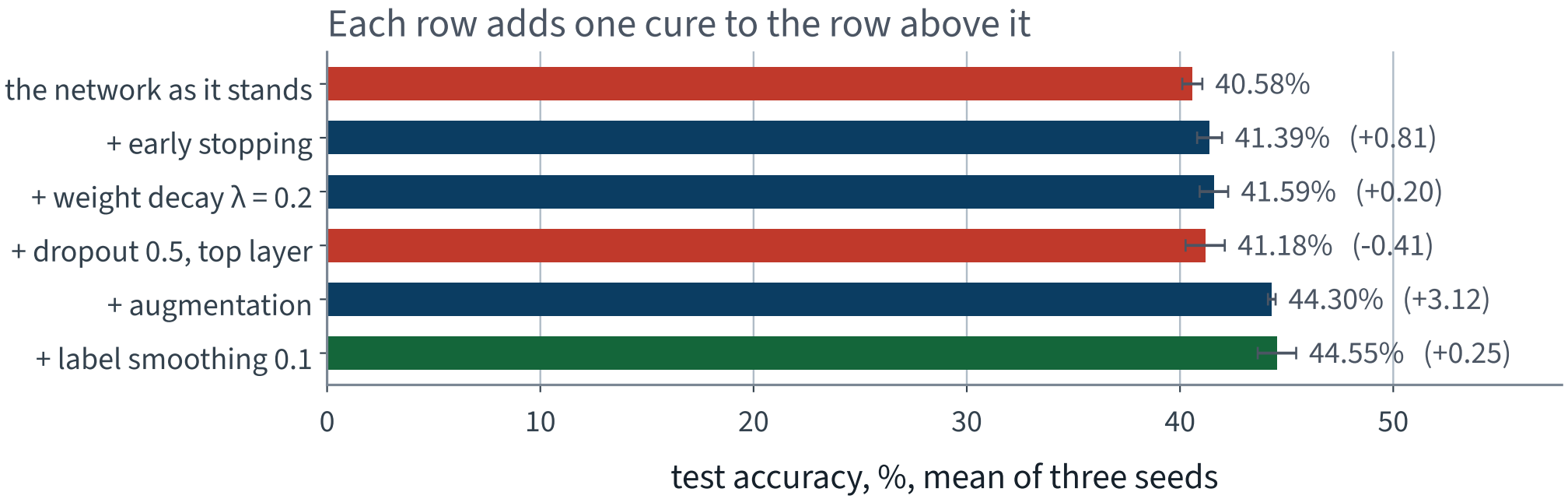
✓ The right panel is what every cure does. The left panel is what you are paid for.

The ladder, continued: three seeds a rung

Configuration	Test accuracy	sd over 3 seeds	Change	Gap
✗ the network as it stands, last epoch	40.58%	±0.47	—	**✗ 38.8**
+ early stopping	41.39%	±0.58	+0.81 (noise)	38.8
+ weight decay, $\lambda = 0.2$	41.59%	±0.67	+0.20 (noise)	34.8
+ dropout 0.5, top layer	41.18%	±0.92	−0.41 (noise)	30.1
✓ + augmentation	44.30%	±0.18	**+3.12 ✓**	**6.5 ✓**
+ label smoothing 0.1	**44.55% ✓**	±0.90	+0.25 (noise)	6.7

Typical spread, the mean of the sd column: **0.62 points**. One rung clears twice that: augmentation, **+3.12**. From the second rung on, every rung is read at its best validation epoch, because early stopping is on the ladder and stays on.

The same ladder, as a picture



✓ Four points from the whole ladder, three of them from one rung — and that rung was the only constraint on the *function* rather than on the weights.

Where the gain came from

	Accuracy points
✓ The largest single step — augmentation	**+3.12 ✓**
The last rung against the first	+3.97
The other four rungs together	+0.85
The gap, first rung to last	38.8 → 6.7

Every cure closed the gap. One of them raised the score.

Which is the answer to “should I add dropout, or weight decay, or stop early?” — yes, cheaply, and do not expect much. The question with a large answer is “what does the network not need to learn?”, and augmentation is how you tell it. Lecture 12 builds that invariance into the architecture instead, and the gain is larger again.

Failure condition — a knob tuned on the test set

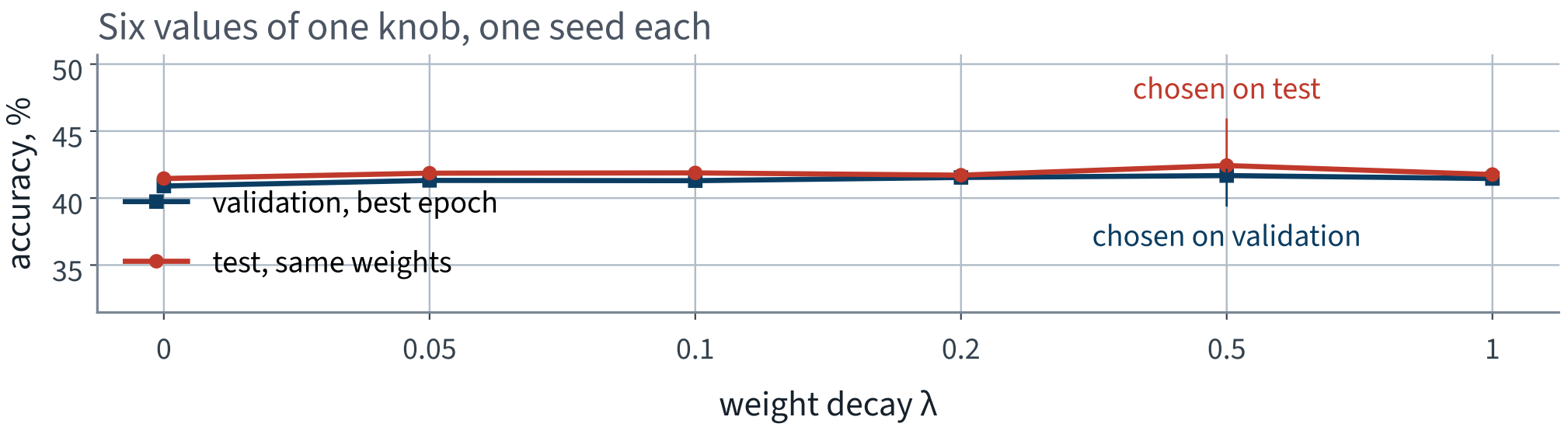
Six values of λ, one seed each, every run read at its best validation epoch. Two ways to choose:

λ	0	0.05	0.1	0.2	0.5	1.0
validation, best epoch	40.90%	41.32%	41.30%	41.54%	**41.68% ✓**	41.46%
test, same weights	41.46%	41.86%	41.88%	41.71%	**42.43% ✓**	41.76%

CHOOSE ON THE TEST SET

The reported number is then the largest of six draws from the test set, and it is optimistic by an amount that *cannot be seen from the number*. Here the two choices happen to coincide, at $\lambda = 0.5$, so the optimism was zero — which you know only because the validation column exists. With a thousand configurations it is the whole result.

The sweep



Six numbers within a point and a half of each other, on one seed. Nobody can read a winner off this plot. The validation set may choose one; the test set may only confirm it, once.

What practitioners actually reach for

Always	Usually	Where it fits
early stopping on a validation metric, best weights kept	AdamW with a small decay, λ around 0.01 to 0.1 for an adaptive optimiser	dropout at 0.2–0.5 on wide, top layers; little on narrow or convolutional ones
augmentation that matches the data's invariances, in the training path only	label smoothing, 0.1	Monte Carlo dropout where a confidence will be acted on

- More data first: the ratio on slide 7 is the cheapest knob of all, and augmentation is the way to turn it when there is no more data
- Then the architecture: a convolution *is* the shift invariance, built in rather than taught (Lecture 12); a pretrained backbone is a thousand hours of someone else's data (Lecture 13)
- Then the five cures here, every knob on the validation set, and the test set read once

The silent-failure catalogue, continued

Symptom	Cause	Lecture
loss flat at $\ln 10$, nothing raised	gradients vanishing by a constant factor per layer	9
training accuracy climbing, validation flat	variance: the network memorises the sample	10
a regularised network at chance	the penalty inside Adam, normalised away	10
dropout makes everything worse	on every layer of a deep stack: the second moment compounds	10
a plausible number, a little low	the training transform on the test loader	10
a published number nobody can reproduce	a knob chosen on the test set	10

Four of today's five raise nothing and print a number. The notebook's answer to all of them is the same: log the training accuracy beside the validation accuracy, and read both.

The notebook

Eighteen cells, on CPU. The repaired network trained for thirty epochs and the two curves; the dropout expectation and the zero-gradient decay step, checked numerically; each cure alone; Monte Carlo dropout with the confidence deciles; the flip-agreement and the augmented-evaluator measurement; the three-seed ladder; and the sweep of λ with both choices printed side by side.

Run first	Then change	And see
Section 5, the plain run	the epochs to 60	the training accuracy keeps climbing; the validation accuracy does not
Section 11, dropout	`top` to 20 at $p = 0.5$	a million on the second moment: the validation curve stays at chance
Section 17, the sweep	add 2.0 and 5.0 to the grid	the validation choice does not move; the optimism cannot fall

The thread count is pinned to four and the device is the CPU, so that two cells can be compared to two decimals. On another CPU the numbers land a point or two away; every conclusion survives that.

Open Lecture 10 in Colab

Design exercise · in the manner of an ML system-design interview

The task: the lesion check inside a dermatology app

> **THE BRIEF**
>
> A consumer health company sells a skin-care app. Its new feature lets a user photograph a mole or lesion with their phone and get one of two answers: *looks fine, check again in a few months*, or *see a dermatologist — here is how to book one*. The model behind it is a classifier trained on photographs labelled by dermatologists. The business measures referrals that turn out to be warranted, users retained, and the cost of each dermatologist visit it triggers.

- **Given:** a few thousand labelled photographs from partner clinics, far fewer than the network has parameters; a stream of unlabelled user photographs once the feature ships
- **Actors:** the product and data teams, the users, the dermatologists who label and who receive the referrals, and the medical-device regulator, because software that tells a person to see or not see a doctor is a medical device
- **Consumers of the output:** the user, the booking flow, and the clinical audit the regulator will ask for

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. Today's method is at the centre: a network with far more parameters than images, and the cures that keep it honest.

Step 1 — Frame it: what is predicted, what it costs to be wrong

- One row per photograph; the label is the dermatologist's verdict, *refer* or *do not refer*; the output is a probability of *refer*, and a threshold on it is the product decision
- The two errors are not symmetric: a missed melanoma is a harm the company answers for in court; an unneeded visit is a cost and an anxious user. The metric is sensitivity at a fixed, high value, with the specificity it buys, read on a reliability diagram
- The baseline: the rule dermatologists already teach — asymmetry, border, colour, diameter — scored on the same photographs

> **PITFALL**
>
> Accuracy. Most lesions are benign, so a model that refers nobody is accurate and lethal, and a model that refers everybody is safe and useless. Fix the sensitivity first, as a clinical requirement agreed with the dermatologists and written into the regulatory file, and optimise everything else under it.

Step 2 — The label is a clinician's opinion, and sometimes a biopsy

Pitfall	What happens	What to do
Two kinds of label	some photographs carry a biopsy result, most carry one dermatologist's visual verdict; the second is noisier and the two are mixed	keep the label source as a column; evaluate on biopsy-confirmed rows; train on both, smoothing the visual ones
Disagreement	two dermatologists disagree on a fraction of borderline lesions; one opinion per image hides that	double-label a slice, measure the agreement, and treat it as the ceiling any model can reach
The clinic's camera	clinic photographs are taken with a dermatoscope under ring light; the user's are a phone in a bathroom	collect a labelled set of phone photographs before shipping, or the training set is of a different task

✓ Write down what a *refer* label means and who gave it. The regulator will ask; so will the lawyer.

Step 3 — Split before anything is fitted, and split by *patient*

- Every photograph of one patient goes to the same side of the split: training, validation or test. Validation chooses every knob; the test side is read once, for the regulatory file
- Stratify by the label and by the clinic, so that no clinic's camera lives only on one side

> **PITFALL**
>
> A random split by photograph puts the same lesion, photographed twice at the same visit, on both sides. The network memorises the patient's skin and the clinic's lighting — today's lecture, exactly — and the validation score is a memory test it passes. The number in the regulatory submission is wrong, and it is discovered in the field, on users.

Step 4 — Augmentation that respects the lesion

- A few thousand images and a network with millions of weights: the ratio on slide 7, worse. Augmentation is the first cure, and the largest
- Flips, rotations by any angle, small crops and small changes of brightness: a lesion has no orientation and a phone has no fixed exposure
- In the training path only. The validation and test photographs are scored as they are

> **PITFALL**
>
> Colour and scale are the signal. A hue shift that would be harmless on a frog turns a benign brown lesion into a suspicious black one; an aggressive zoom changes the diameter that the clinical rule is built on. The transforms must be chosen with a dermatologist in the room, and each one tested on the validation side for whether it helps or hurts.

Step 5 — The model, and the cures on it

- A pretrained convolutional backbone, fine-tuned — the subject of the next three lectures — with a dropout layer before the head
- AdamW with a small decay; early stopping on the validation sensitivity at the agreed specificity, the best weights kept
- Label smoothing on the visually-labelled rows, where the label is an opinion

> **PITFALL**
>
> A training curve that reaches perfect accuracy on the clinic photographs. With a few thousand images the network will memorise them within a few epochs, and a team that watches only the training loss ships the last epoch. Log the validation curve beside it from the first run, stop on it, and keep the ablation table — which cure bought what — because the regulator’s reviewer will ask why each component is there.

Step 6 — When the model is not sure, it says so

- Monte Carlo dropout at inference: twenty passes, the mean as the probability, the spread as the confidence
- Three outputs, not two: *looks fine*, *see a dermatologist*, and *this photograph cannot be assessed — retake it, or see a dermatologist*, for the least confident fraction
- A separate, cheap check that the photograph is of skin at all, in focus, and lit; most low-confidence cases are bad photographs

> **PITFALL**
>
> A two-answer product forces a confident verdict on a photograph the model has never seen the like of — a skin tone absent from the clinic data, a tattoo, a bandage. The least confident decile is where the accuracy is lowest, as it was in the lecture, and it is also where the users who most need a doctor are. Abstain and refer; the cost of a visit is bounded, the cost of a confident wrong *looks fine* is not.

Step 7 — The threshold, the calibration, and who signs it

- The sensitivity target fixes the threshold on the validation side; the test side, read once, goes into the submission with its confidence interval
- Calibrate the probability on a held-out slice so that “eight in ten” means eight in ten, and check it per skin tone, per clinic and per phone model — a model calibrated on average and wrong for one group is a model that is wrong
- The threshold is a clinical decision, signed by the medical director, not a hyperparameter

PITFALL — TUNING ON THE TEST SET

The team tries a dozen backbones and thresholds and reports the best test sensitivity. That number is the largest of a dozen draws, and the regulator’s own evaluation on fresh patients comes in lower. The submission is rejected, or worse, accepted. One test set, read once, by someone who did not choose the model.

Step 8 — Release, monitor, retrain

- Release behind a clinical safety case: a limited cohort first, every referral followed up, every *looks fine* sampled and reviewed by a dermatologist
- Monitor the input distribution — phone models, skin tones, lighting, the abstention rate — and the referral rate against the rate the business planned for
- Retrain on the new, phone-taken, dermatologist-reviewed photographs, through the same split-by-patient pipeline, and re-evaluate on the untouched test set before every release; a changed model is a changed device

> **PITFALL**
>
> The feature changes who photographs what. Worried users send more pictures of ordinary moles; reassured users stop sending the ones that changed. The labels that come back are from the referrals the model chose, so retraining on them teaches the model its own habits. Keep the sampled review of the *looks fine* answers — that is the only unbiased signal the product produces.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Clinic photographs, label source kept → split by patient → augmentation agreed with a dermatologist, training path only → pretrained backbone, dropout, AdamW, early stopping on validation sensitivity → Monte Carlo dropout and a third answer → threshold signed off, test set read once → audited release → monitor, retrain, re-evaluate.

Pitfall	Address it by
Accuracy as the metric	sensitivity fixed first, as a clinical requirement
Clinic labels, phone photographs	a labelled phone set before shipping; the label source kept
A split by photograph	a split by patient, stratified by clinic
Augmentation that alters the signal	transforms chosen with a dermatologist, tested on validation
Shipping the last epoch	the validation curve from the first run; the best weights kept
A confident verdict on an unseen kind of image	Monte Carlo dropout; abstain and refer
The best of a dozen test scores	one test set, read once, by someone else

What we did

1. Trained Lecture 9's repaired network for thirty epochs and watched it memorise: 78.05% on its training images, 38.90% on validation, a gap of thirty-nine points — variance, with 504,210 parameters for 10,000 images
2. Derived dropout as an average over 2^n thinned networks: $\mathbb{E}[\tilde{z}] = (1-p)\,z$, so divide by $1-p$ in training and the full network computes the expectation; the second moment grows by $1/(1-p)$ per dropout layer, which is why one layer works and twenty do not
3. Derived weight decay as the shrinkage $(1-\eta\lambda)\,\theta$ before every step, and saw Adam normalise the coupled penalty into a network at chance; AdamW applies the shrinkage outside the step
4. Measured five cures alone and on a three-seed ladder: every one narrowed the gap, and augmentation alone raised the test accuracy beyond the seed spread, +3.12 points, because it constrains the function rather than the weights
5. Kept the masks on at test time — Monte Carlo dropout — and turned confidence into a decision: answer the surest 80%, refer the rest
6. Named the failure nobody can see from the number: a knob chosen on the test set; the validation set chooses, the test set is read once
7. Designed the lesion check inside a dermatology app in nine steps — sensitivity fixed as a clinical requirement, labels with their source, a split by patient, augmentation agreed with a dermatologist, early stopping and dropout on a pretrained backbone, a third answer for the unsure cases, a threshold someone signs, a release that is audited — each step named for the pitfall it avoids

In the book

Topic	Chapter
Overfitting, and avoiding it through regularisation	11
ℓ_1 and ℓ_2 regularisation; AdamW	11
Dropout, and Monte Carlo dropout	11
Early stopping	11
Data augmentation	12
The train / validation / test split, and why the test set is read once	2

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 10

five, in the style of the written paper

Exercises — Lecture 10

1 A hidden unit's activations a_j feed a pre-activation $z = \sum_j w_j a_j$. Dropout keeps each unit with probability $1 - p$. Derive $\mathbb{E}[\tilde{z}]$ under the mask, and state what inverted dropout divides by so that the network run without masks computes that expectation. [4]

2 Dropout preserves the expected pre-activation. Show that it does not preserve the second moment, give the factor per dropout layer, and say what that implies for a stack of twenty layers. [5]

3 Add $\frac{\lambda}{2}\|\theta\|^2$ to the loss and write one gradient-descent step. Then say what happens to that term when it is handed to Adam as part of the gradient, and how AdamW differs. [4]

Solutions on Lecture 11's deck.

Exercises — Lecture 10 (continued)

4 A colleague trains for fifty epochs, keeps the weights of the epoch with the highest validation accuracy, and reports that validation accuracy as the model's result. Name the two things that are right about this and the one that is wrong. [4]

5 Six values of a weight-decay knob are tried, each run read at its best validation epoch. The value with the highest validation accuracy is reported on the test set, giving t_v; the value with the highest test accuracy is reported, giving t_t. State the sign of $t_t - t_v$, when it can be zero, and why the size of the bias cannot be read from either number. [4]

Solutions on Lecture 11’s deck.

THE FIVE SET AT THE END OF LECTURE 9

Solutions Lecture 9

not part of this lecture

Solutions — Lecture 9

1. One layer multiplies the standard deviation of the backward signal by…

✓ They multiply it by ρ^{L-1} — $L-1$ steps between the first gradient and the last — so anything but 1 vanishes or explodes geometrically.

- A constant applied L times is exponential in L
- There is no regime where a repeated constant other than 1 is safe

2. Preserving the forward signal asks for $\mathrm{Var}(w) = 1/n_{\text{in}}$ and preserving the gradient…

✓ They agree only when $n_{\text{in}} = n_{\text{out}}$; Glorot takes the harmonic mean, $2/(n_{\text{in}} + n_{\text{out}})$.

- Any layer that changes width cannot satisfy both
- The compromise accepts a small error in each direction rather than a large one in either

Solutions — Lecture 9 (2)

3. He initialisation doubles the weight variance for ReLU. Derive the factor of two in one line.

✓ ReLU zeroes half of a symmetric zero-mean input, so $\mathbb{E}[\mathrm{ReLU}(z)^2] = \frac{1}{2}\mathbb{E}[z^2]$.

- Exactly half the variance is discarded at each layer
- Doubling $\mathrm{Var}(w)$ puts it back

4. A gradient profile that is a straight line on a log axis tells you something a curved one does not. State…

✓ The attenuation is the same factor at every layer, so it is the initialisation rather than one bad layer.

- A constant ratio per layer is a straight line in log space
- A single broken layer would show a step, not a slope

Solutions — Lecture 9 (3)

5. Gradient clipping is applied to a network whose gradients are fifteen orders of magnitude too small. Predict…

✓ No effect — clipping bounds gradients that are too large.

- The threshold is never reached, so nothing is rescaled
- Reaching for it means the failure mode was never measured

LECTURE 11 · LECTURE NOTES

Autoencoders and representation learning

A network with no labels: compress an image to a code and reconstruct it
A linear autoencoder recovers the principal subspace, derived
The code as a representation: retrieval, clustering, anomalies

Run the Lecture 10 notebook first